Secure Delivery Defect and Station Performance Dashboard

SWE-550: Software Engineering & Security Principles

Omar Soto Palos

Instructor: Shane Carroll

November 10, 2026

V6

Project Overview

For Milestone 6, I wanted to show how I would get my Flask dashboard ready for a secure release, not just how it works on my laptop. In the earlier milestones, I worked on login security, user roles, session timeouts, and security headers. I also used Bandit and OWASP ZAP to look for security issues. This milestone brings those pieces together and shows what I would check before releasing the application.

I set up GitHub Actions and confirmed that my automated checks passed. It installs my dependencies, checks the Python files, runs five security tests, scans the code with Bandit, and checks the dependencies with pip-audit. I have not deployed the dashboard to production. The packaging, staging release, approval, and verification steps in this report explain what I would do when the project is ready for deployment.

Task 1: Secure Deployment Workflow and Risk Controls

Deployment artifact: ordered release workflow

Order	Stage	Trigger, control, and expected evidence
1	Build	Push or pull request to main, or manual workflow_dispatch. Check out source, set up Python 3.14, install requirements, compile Python files. Evidence: CI job log.

2	Test	Run pytest, Bandit, and pip-audit. If a required check fails, I will stop the release review. Evidence: passing GitHub Actions logs.
3	Package (planned)	From an approved commit, prepare a versioned source archive; exclude .env, record version, full commit SHA and SHA-256 checksum.
4	Deploy (planned)	Release owner reviews tests, residual risks, and package; records approval, operator deploys to controlled staging with separately configured secrets.
5	Verify (planned)	Check startup, login, role restrictions, security headers, logs and deployed commit; reject release or roll back on failed criteria.

My workflow starts when I push changes to main, open a pull request to main, or start it manually. GitHub Actions runs the checks on Ubuntu, while I use Flask locally for development. My next deployment target would be a controlled staging environment, with production coming later. For a release, I would create a versioned package and record the Git commit and checksum, so I know exactly what was deployed. For this project, I would be responsible for the release; on a larger team, the approval and deployment duties could be assigned to different people.

Before approving a staging release, I would check that the code was reviewed, the CI tests passed, no secret values were included, and any remaining risks were reviewed. I would also record the approval. Right now, that is a documented step, not an approval control that GitHub automatically enforces. Since the workflow runs after a push to main, a passing check also does not stop someone from pushing code in the first place.

How the workflow addresses earlier risks:

Earlier risk	Deployment control	How it helps
Broken access control	Five pytest role-boundary tests in GitHub Actions	My tests check that users who are not logged in are redirected, team members cannot reach manager records, and managers cannot reach admin functions. A failure blocks release approval under this plan.
Accidental secret exposure	Environment-based SECRET_KEY, GitHub	Keeps the test secret separate from source and reduces the

	repository secret, .gitignore, pre-release source/history review	chance of an accidental commit. Git ignore is not a history scanner; dedicated secret scanning remains planned.
Known vulnerable dependencies	pip-audit requirements.txt	Checks the pinned requirements against known vulnerability advisories at the time of the run, failure prompts review before release.

Current implementation boundary

I saved the automated workflow in .github/workflows/security-checks.yml and the deployment instructions in docs/milestone-6/deployment-workflow.md. The workflow currently runs the build and security checks. It does not automatically package the application, deploy it to staging, or run post-deployment verification. Those are the next steps in my release plan.

Task 2: Automated Build, Dependency, and Security Checks

Configuration and execution

I created a GitHub Actions workflow named Secure Dashboard CI. It runs when I push changes, open a pull request, or start it manually. The workflow sets up Python 3.14, installs the required packages, checks the Python files, and runs pytest, Bandit, and pip-audit. I also created

tests/test_security.py. Those tests use fictional users and a temporary database so I can check permissions without changing the delivery records in my local dashboard.

Check / command	Observed result	Pass / fail criterion
python -m compileall -q app.py models.py	GitHub Actions step passed	Pass if command exits 0; fail on syntax/compilation error.
python -m pytest -q	5 passed	Pass if all five expected tests pass; fail if any test fails.
python -m bandit -r app.py models.py	398 lines scanned; no issues identified	Pass if scan exits 0 with no findings under the current configuration; otherwise investigate.
python -m pip_audit -r requirements.txt	No known vulnerabilities found	Pass if audit exits 0; fail if known vulnerability findings or check error require review.

The developer is responsible for keeping the workflow updated, and GitHub Actions runs the checks and saves the results. I reviewed the Build, Test, and Security Checks job, which finished successfully in 36 seconds. These results show that the specific checks I ran passed. They do not mean every possible vulnerability has been found. The workflow file and the screenshots in Figures 1–4 show what I ran and what happened.

Role-based access tests run in CI

Scenario	Expected result	Observed test
Unauthenticated request to /dashboard	302 redirect	Passed
Team member to /manage-records	403 Forbidden	Passed
Manager to /manage-records	200 OK	Passed
Manager to /admin	403 Forbidden	Passed
Administrator to /admin	200 OK	Passed

This is important to me because my professor previously asked how I could prove that a future code change would not accidentally remove an access-control fix. Now I have five automated tests I can run again whenever I update my project, instead of only using screenshots from one manual test.

Task 3: Secrets Handling and Environment Configuration

One of the main things I needed to protect was my Flask SECRET_KEY. My application reads it from an environment variable and will not start if the key is missing. On my laptop, I keep the key in a local .env file. For GitHub Actions, I created a different secret named CI_FLASK_SECRET_KEY and made the workflow pass it to Flask as SECRET_KEY. I did not put either actual secret value in this report.

Setting	Purpose	Where configured / safe example
SECRET_KEY	Signs Flask session data and protects session integrity	Local .env or approved deployment secret store; placeholder: <generate-unique-random-key>
CI_FLASK_SECRET_KEY	Separate secret used when running automated CI tests	GitHub Settings > Secrets and variables > Actions; never place value in workflow YAML
SESSION_COOKIE_SECURE	Controls Secure flag on session cookies	Current app.py setting: False for local HTTP. Must be True with HTTPS before public deployment.
Python / requirements	Controls runtime and reproducible dependencies	Python 3.14 selected in .github/workflows/security-checks.yml; packages pinned in requirements.txt

I also checked my .gitignore file and confirmed it excludes .env and database files. When I ran git ls-files .env dashboard.db, nothing was returned, which means those two files were not tracked in the current Git index. That is a good check, but it does not tell me whether a secret was

ever committed in the past. Checking Git history and adding an automatic secret scanner are improvements I still want to make.

Setup for another authorized team member

1. Obtain authorized access to the private repository and clone the approved commit.

2. Create a Python virtual environment and install dependencies using pip install -r requirements.txt.

3. Create a local .env file outside Git tracking; set SECRET_KEY to a freshly generated random value. Never copy the value into screenshots or source code.

4. Confirm the application starts locally and run python -m pytest -q.

5. For CI administration, an authorized repository administrator creates CI_FLASK_SECRET_KEY under GitHub Actions secrets; the workflow references the name, not the value.

6. Review git status and staged changes before committing. If a key is exposed, rotate it, investigate exposure, and review the relevant history.

I put the full setup directions in docs/milestone-6/environment-setup.md. Before I use this application outside my local testing environment, I would also need HTTPS, secure session cookies, a production-ready server, and properly restricted access to secrets. I have not tested those production settings yet.

Task 4: Release, Rollback, Verification, Remediation, and Monitoring

Release identity and approval

Before deploying it, I would record the complete Git commit, package checksum, environment, release date, and approval. The release owner would make the decision, and the

deployment operator would carry it out. Since this is my course project, I would document both responsibilities. I have not completed a staging deployment yet.

When I would stop or roll back

I would stop or roll back a release if the application would not start, login stopped working, an unauthorized user could get into restricted pages, important security headers were missing, or major errors appeared. I would also stop immediately if a secret was exposed. In that case, going back to the old code would not be enough; I would need to replace the exposed secret and investigate what happened.

Ordered rollback procedure (planned)

1. Stop the rollout and record the failing verification or monitoring signal.

2. Notify the release owner and approve the rollback decision.

3. Identify the last verified version and its commit/package checksum.

4. Preserve error logs and the current database before making changes.

5. Restore the previously approved application package and previous approved configuration when configuration changed. Do not restore a compromised secret.

6. Restore database data only if a release included an incompatible schema/data change and a verified backup is needed; ordinary code-only rollback requires no database restoration.

7. Restart the application and repeat functional and security verification checks.

8. Log the outcome, open a remediation issue, and require review before another release attempt.

Post-deployment verification checklist

Check	Pass criterion	Fail / action
Startup / login	Service starts; GET / returns 200	Investigate startup/configuration; hold release.
Authorized dashboard	Manager GET /dashboard returns 200	Hold release; inspect authentication/session logs.
Team member role boundary	GET /manage-records returns 403	Rollback/block release; investigate authorization.
Manager admin boundary	GET /admin returns 403	Rollback/block release; investigate authorization.
Headers	CSP, X-Frame-Options: DENY, X-Content-Type-Options: nosniff are present	Fix response middleware; do not approve release.
Release identity / secrets	Deployed commit equals approved SHA; no secret in package/logs	Stop, correct artifact, rotate if exposed.

Monitoring, response, and escalation

After a future staging release, I would check the startup and error logs, failed requests, login problems, rate-limit events, session timeouts, and the GitHub Actions results for the release commit. I would keep a record of what I checked. If something failed, I would review the logs, stop the

release if necessary, fix the problem or roll back, and test again. Serious security issues would be escalated to the application owner or security team in a real work environment.

I also need to keep two findings from Milestone 5 in mind. My CSP still allows inline styles, and the local Werkzeug server still reveals version information. I am not saying I fixed those problems. I would review them and complete the appropriate hardening before approving a public production release.

Task 5: Submission Evidence and Repository References

I included the deployment steps, successful GitHub Actions results, instructions for keeping secrets protected, and my release and rollback plan in this report. I used the screenshots from my GitHub Actions job so the results can be checked. Each screenshot has a caption and a short explanation of what I tested and why it matters for this milestone. I did not include any real secret values.

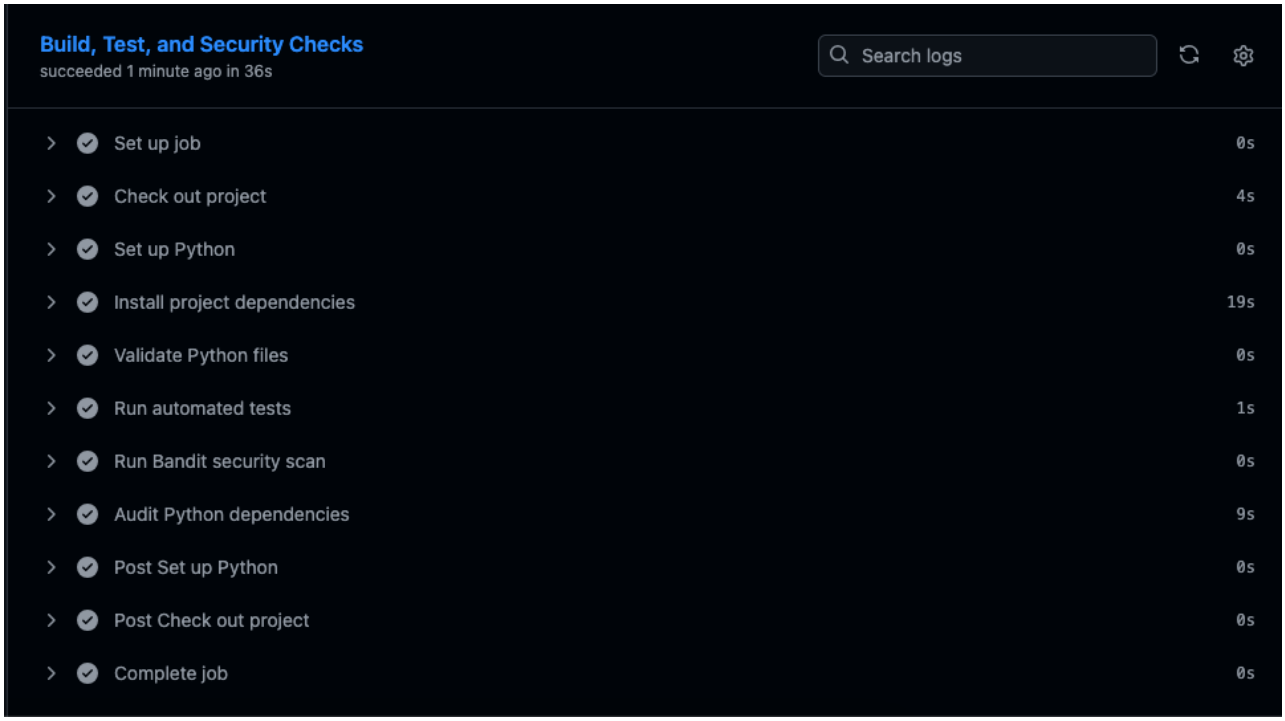
Repository item	Path
CI workflow	.github/workflows/security-checks.yml
Automated security tests	tests/test_security.py
Deployment workflow	docs/milestone-6/deployment-workflow.md
Environment guide	docs/milestone-6/environment-setup.md
Release/rollback guide	docs/milestone-6/release-and-rollback.md
Evidence screenshots	docs/screenshots/M6-01... through M6-04...

Repository URL: https://github.com/Ramo3695/Secure-Delivery-Defect-Dashboard/tree/main

I added the four evidence images to GitHub through the website. I have not independently rechecked that remote file list while preparing this report, so I would verify the repository before submitting. I also want to be clear that GitHub Actions completed the checks, but I have not yet packaged a staging release, approved a deployment, or performed an actual rollback. Those steps are documented as my release plan.

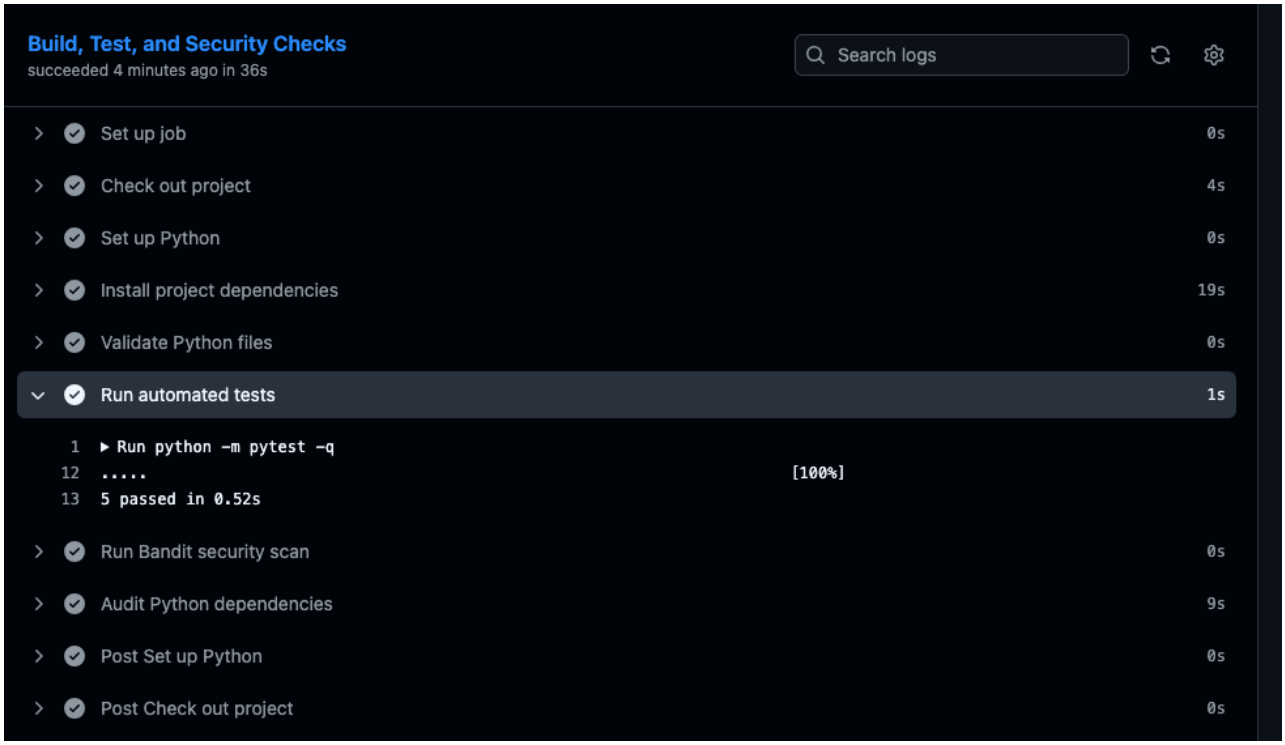
These four screenshots come from my successful GitHub Actions job. I included them to show the actual commands and results rather than just saying the checks worked.

Figure 1. Completed GitHub Actions job



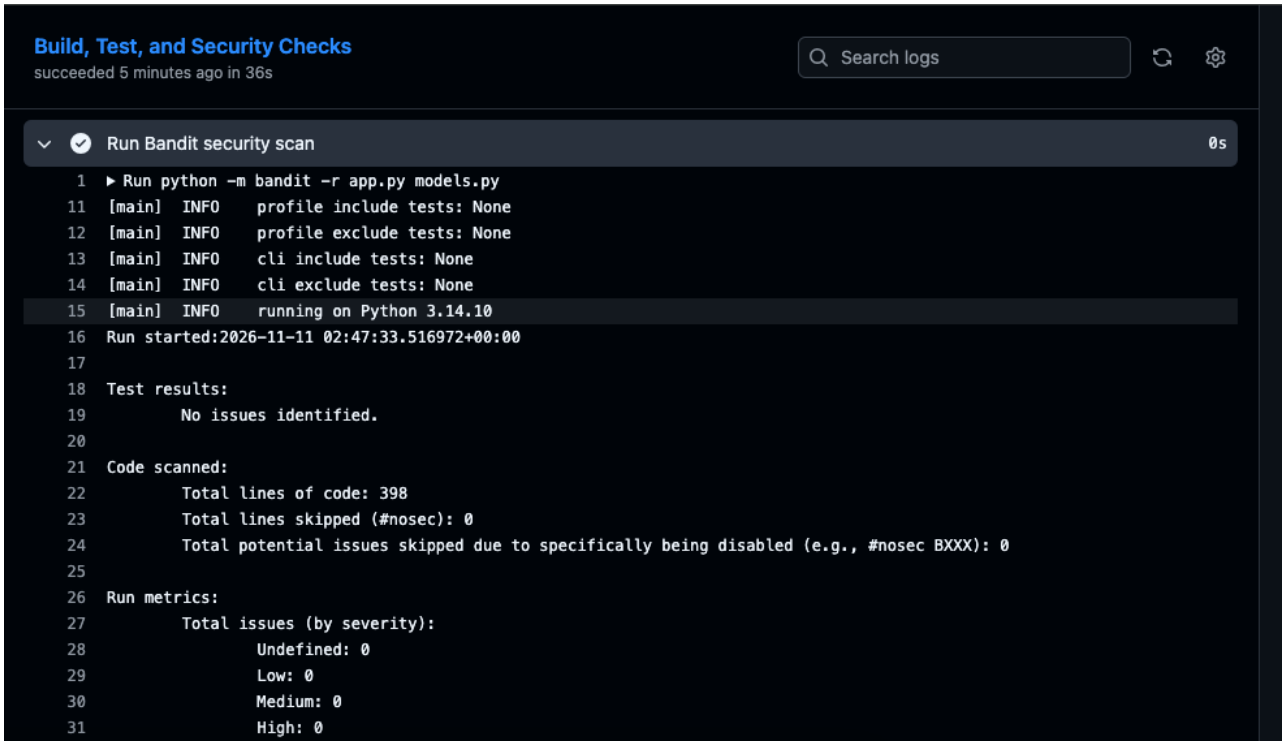
This screenshot shows my Build, Test, and Security Checks job completing successfully in 36 seconds. Dependency installation, Python validation, pytest, Bandit, and pip-audit all passed. It gives me proof that the automated checks actually ran for Task 2.

Figure 2. Automated access-control tests



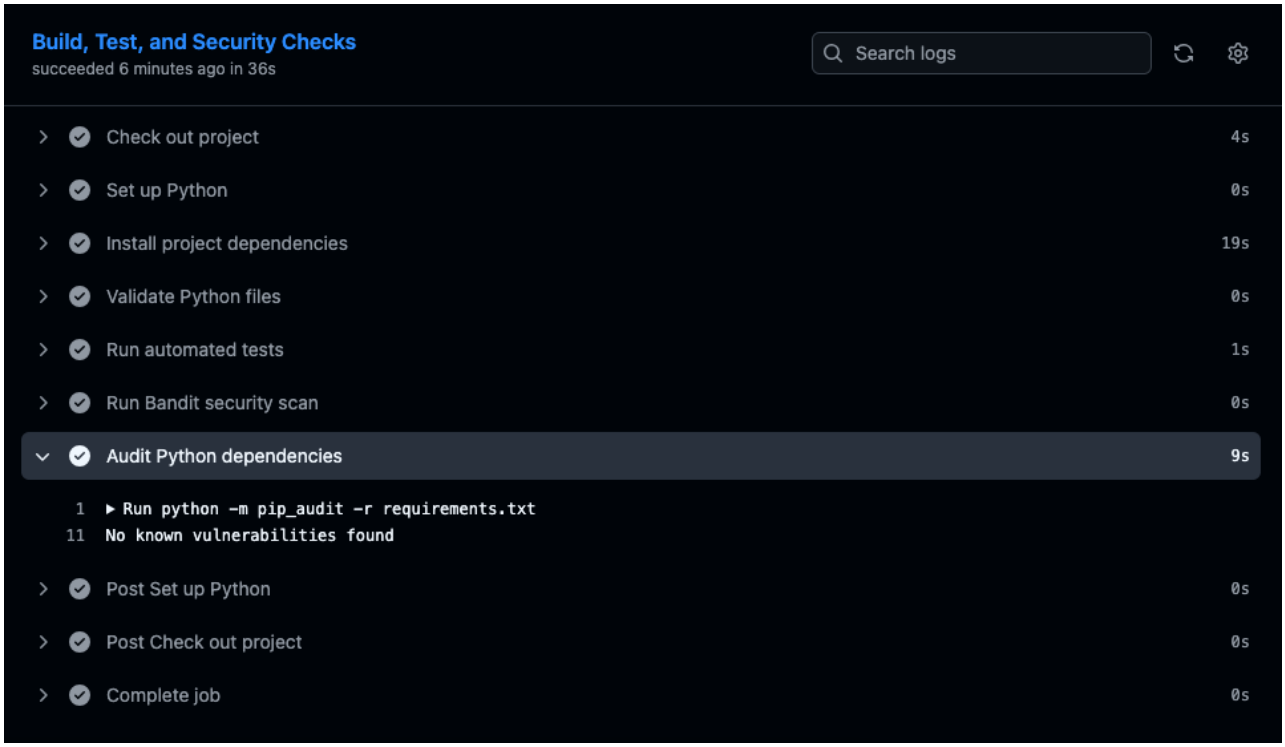
GitHub Actions ran python -m pytest -q, and all five security tests passed in 0.52 seconds. These tests check the access rules for users who are not logged in, team members, managers, and administrators. This is the automated role-checking evidence required for Task 2.

Figure 3. Static-analysis security check



I ran Bandit through the workflow using python -m bandit -r app.py models.py. It checked 398 lines of code and reported zero issues, with no lines skipped. This is evidence that the static scan passed, although it cannot catch every kind of security issue.

Figure 4. Dependency vulnerability check



The workflow ran python -m pip_audit -r requirements.txt and reported no known vulnerabilities in the listed dependencies at the time of the check. This helps me catch known dependency problems before approving an update.

Final Reflection

This milestone showed me that having a working application is not the same as being ready to release it. I now have GitHub Actions checking my code, user permissions, and dependencies whenever I make changes. I also wrote down how I would protect my secret keys, approve a release, check the application after deployment, and roll back if something went wrong. My next steps would be to add automatic secret scanning and packaging, then test the staging deployment before considering a production release.